with local and session storage, and, perhaps the most promising of all, service worker caching.

7.9 Summary

- **Caches** are ubiquitous at all levels of the computation stack, from hardware caches in your processor to distributed web caches on the Internet.
- **Composite data structures:** we have seen a different approach to solving problems, using data structures as building blocks to create complex systems, and we took a journey that led us to discover many fundamental laws, principles and designs of computer science and engineering.
- **Do not compute things twice**: especially if they are expensive to compute. Whether this means running complex vector algebra on a GPU or connecting to an external subscription service, if it is reasonable to assume that the result you just computed might be used again in the near future, just store it in a cache.
- **Cache and web applications:** caches are useful at all levels, but while you could probably ignore them in standalone applications running locally on a single computer, they became paramount to allow web applications to scale to millions or billions of requests per day.
- **Eviction policy:** there are different types of caches, and the distinction is based on their eviction policy: a cache has usually a fixed maximum size, and when reached, the eviction policy states which entries must be purged out of the cache to make room for the newer ones.